`spi-pipeline` — five threads, eight semaphores, two partitions, one CRC

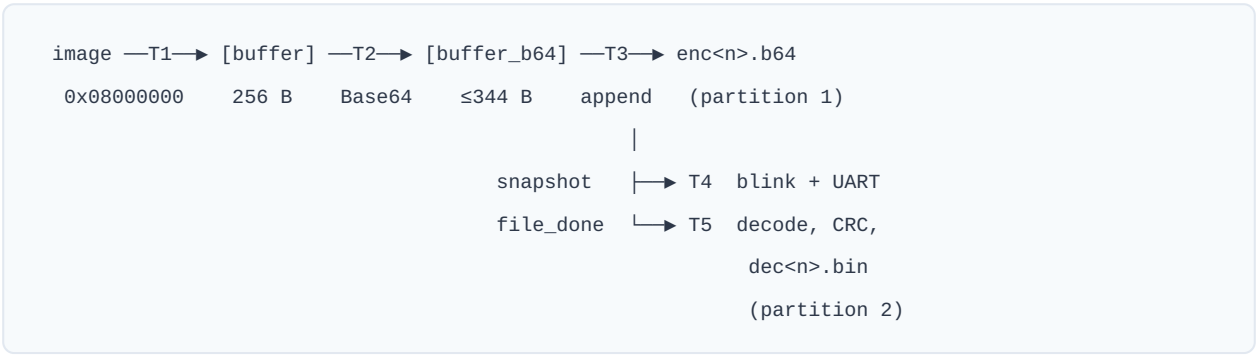
The **C++** port of the application specified by `docs/Rust/weact-spi.md` in the predecessor tree (`luckfox_lyra_micro-os`), running on the **WeAct Studio F411CE** under the shared **micro-os-plus-iii** kernel and the upstream **Cortex-M port**.

This is the C++ sibling of the Rust example of the same name (`micro-os-plus-iii-cortexm-weactf411-rust/examples/spi-pipeline`). The Rust port and this one implement the same specification with the same shapes — five threads, a 256-byte raw slot, a Base64 slot, eight semaphores, two partitions — and the same verdicts. What differs is the kernel underneath: here every primitive is a **C++ µOS++ object** (`os::rtos::thread`, `semaphore_counting`, `mutex`, `mutex_recursive`, `sysclock`), the source of truth for the concurrency rules is the µOS++ kernel source itself, and the whole thing is an `arm-none-eabi` CMake build, made by the harness, rather than a Cargo one. This README is about *that*: what the port does, the µOS++ techniques it relies on, and the solutions this specific combination — a five-thread RTOS app driving SPI-NOR flash through a DMA/IRQ/polled driver — forced us to reach for.

The board it runs on	`test/boards/weactf411/` in this repository (BSP, linker script, `openocd.cfg`, `hw.sh`)
The kernel	`micro-os-plus-iii-smp.git`
The Cortex-M port	this repository, `micro-os-plus-iii-cortexm.git`
The Rust original	`micro-os-plus-iii-cortexm-weactf411-rust/examples/spi-pipeline`, in the predecessor tree
The spec + its fixes	`docs/Rust/weact-spi.md`, in the predecessor tree (`luckfox_lyra_micro-os`)

What it does

Five threads move the **application image itself** through a Base64 encoder onto a flat-file archive on the SPI NOR flash, while a fourth thread watches the file grow from the UART as the LED blinks, and a fifth reads the whole thing back, decodes it onto a **second partition**, and compares the CRC-32 against the source. Eight times (`ITERATIONS`).



Thread	Role	Waits on	Posts	Ends when
T1	image → raw slot	`g_buf_empty`	`g_buf_full`, `g_copy_done`	the window is exhausted
T2	Base64 raw slot → encoded slot	`g_b64_empty`, `g_buf_full`	`g_buf_empty`, `g_b64_full`	it forwarded the final chunk
T3	append to `enc<n>.b64`	`g_b64_full`, then `g_copy_done`	`g_b64_empty`, `g_write_snapshot`, `g_file_done`	final chunk **and** `chunks` seen
T4	blink (0.3 s) + snapshot to UART	`g_write_snapshot` (bounded)	—	`BLINK_CYCLES` cycles
T5	read back, decode, CRC	`g_file_done`	`g_verify_done`	it printed its verdict

`os_main` (the kernel's `main` thread) resets the pipeline state, spawns the five, waits for `g_verify_done`, `join()`s each, and checks the verdict record. Every check is **in-program** and printed as one of six `pipe N` lines, then `spi-pipeline: PASS` or `FAIL, n`, and last the harness's `RESULT: PASS` or `RESULT: FAIL`.

Layout

`main.cpp`	the five thread bodies, the outer loop, the six verdicts
`pipeline.hpp/.cpp`	sizes, the two hand-off slots, the eight semaphores, the verdict record, the console
`base64.hpp/.cpp`	RFC 4648 §4 with **per-chunk** padding, and the §10 vectors it checks itself against at start-up
`store.hpp/.cpp`	`Storage`: the two partitions + the writer, and the RAM backend QEMU takes
`check.sh`	asserts the transcript's gates on either machine

From the board, `test/boards/weactf411/`:

`src/spi.cpp`, `include/spi.hpp`	SPI1 in three transports — DMA2, SPI1 IRQ, polled — plus `probe()`
`src/flash.cpp`, `include/flash.hpp`	W25Q SPI NOR driver
`src/flatfs.cpp`, `include/flatfs.hpp`	the archive: `Partition`, `Writer`, CRC-32 (shared with the BSP)
`src/os-hooks.c`	the clock boot
`openocd.cfg`, `hw.sh`	the probe (a CMSIS-DAP DAPLink) and the port's own runner

The harness builds the application from this folder: the port's test builder makes one application per directory, and `tests.cmake` adds what it needs.

The µOS++ techniques this app is built on

1. Threads are objects; a returning thread is a joinable one

µOS++ threads are constructed, not spawned: you build a `os::rtos::thread` with a `void* (*)(void*)` body and `thread::attributes`, and the scheduler starts it on construction. A thread whose body **returns** is a normal thing: the kernel's `internal_invoke_with_exit_` calls your function and then `internal_exit_`, which unlinks the thread, parks it on the kernel's *terminated* list, and yields. The **idle thread** then reaps it — `os_rtos_idle_actions` walks the terminated list and calls `internal_destroy_`, which frees a dynamically allocated stack and flips the state to `destroyed`. `thread::join()` is precisely *"wait until state is destroyed"*.

That is why this example can create and join five threads per iteration, eight times over, without leaking:

```
thread::attributes attr = thread::initializer;
attr.th_stack_size_bytes = pipeline::STACK;   // heap stack, no address given

thread* t1 = new thread{ "t1", t1_image_to_buffer, (void*)src_len, attr };
// … t2 … t5 …

pipeline::g_verify_done.wait ();   // T5's signal that all five are done
t1->join (); t2->join (); …        // wait until each is state::destroyed
delete t1; …                       // now safe: the object is no longer live
```

Two µOS++ details made this the *right* shape rather than an accident:

- **Stacks come from the RTOS heap when you give no static address.** The thread attributes fall back to the kernel's default allocator, so a heap stack is freed by `internal_destroy_` when the idle thread reaps the thread. The per-iteration `heap free` witness (a semihosting line after each spawn set) stays flat across all eight iterations — that is the reaper, working.

- `join()` **returns only after** `destroyed`**.** So `delete` after `join` is never a use-after-free of a *running* thread, and the destructor (`~thread`) sees `state_ == destroyed` and skips its `kill()`. The Rust original models this with `try_spawn` + drop; C++ models it with `new` + `join` + `delete`.

Why not static stack arrays (as `mos-test1` does)? Because eight iterations of five threads would need either 40 static arrays or reuse-with- restart, and µOS++ threads are single-shot objects. Heap stacks let the idle reaper do the recycling, which is exactly what the Rust port's reaper does too — the two kernels arrived at the same design independently.

2. The eight semaphores are `semaphore_counting`, max 1

µOS++ offers `semaphore_binary` and `semaphore_counting`. The specification's table lists two *counting* semaphores (the "empty" pair, which must be able to hold a token across a reset) and six *binary* ones. The port implements **all eight as** `semaphore_counting{ name, 1, initial }`, i.e. counting semaphores whose ceiling is 1:

```
semaphore_counting g_buf_empty{ "buf_empty", 1, 1 };   // starts with a token
semaphore_counting g_buf_full { "buf_full",  1, 0 };
```

This is deliberate, and it mirrors the Rust implementation exactly: a binary semaphore's `post()` on an already-full semaphore must not corrupt anything, and µOS++ `post()` returns `result::EAGAIN` when `count_ >= max_value_` rather than overflowing. Because every one of these has `max == 1`, a "second post while full" is a *dropped, reported* event, never a counter that climbs. The code relies on that in two places:

- T3 posts `g_write_snapshot` after every chunk even when T4 has not yet taken the previous token — `EAGAIN` there means "T4 is mid-blink, not new information", and is deliberately discarded;
- `pipeline::reset()` **drains and refills** each semaphore to an exact known state before an iteration: `refill()` = `drain()` then one `post()`, which puts an empty `g_buf_empty` / `g_b64_empty` back to exactly one token.

The refill is load-bearing in a way a fresh reader would not guess: the initial `1` in the constructors is **not** what prevents the first-iteration deadlock. `reset()` runs before every iteration *including the first* and sets the counts itself. Mutating the constructor value to `0` changes nothing; deleting the `refill()` line hangs the run on its first chunk. The Rust plan documents this as spec-fix #7 and mutation #4; the C++ port keeps the same invariant in `pipeline.cpp`.

3. `reset()` runs before the threads, and owns the per-iteration state

The semaphores, the two slots and the verdict record are globals that outlive a single iteration. µOS++ constructs them once at static-init time, so the app cannot rebuild them per iteration. `pipeline::reset()` puts everything back where an iteration expects it:

```
void reset ()
{
  refill (g_buf_empty);  refill (g_b64_empty);
  drain (g_buf_full);  drain (g_b64_full);
  drain (g_copy_done);  drain (g_file_done);
  drain (g_verify_done);  drain (g_write_snapshot);

  const std::uint32_t crc0 = g_run.crc0;        // crc0 is a property of the
  std::memset (&g_run, 0, sizeof (g_run));      // image, not of an iteration:
  g_run.crc0 = crc0;                            // it survives the reset
  g_run.monotonic = true;                       // Run::new() starts it true

  g_raw.len = 0;  g_raw.last = false;
  g_enc.len = 0;  g_enc.last = false;
}
```

The subtle line is `g_run.monotonic = true`. The Rust `Run::new()` initialises `monotonic: true`, and the C++ port's first draft `memset` the whole record to zero — leaving `monotonic == false` — and then T4 folded its observation with `g_run.monotonic = g_run.monotonic && this_cycle_monotonic`. The result was a **permanent false**: pipe 4 failed on every iteration even though every individual cycle was monotonic. It was a two-line fix (re-seed `monotonic` in `reset()`), found by reading the *Rust* verdict record and noticing the C++ `Run::new()` had silently lost its initialiser. The lesson is the one this whole port repeats: when two ports share a spec, the C++ object initialisers must reproduce the Rust `const fn new()` defaults exactly, or a boolean that "obviously" starts false will quietly invalidate a whole verdict class.

4. The console is a `mutex_recursive`, because the banner re-enters

Every UART line goes through `pipeline::line()`, which takes `g_console` (a `os::rtos::mutex_recursive`) for the duration of the write so two threads cannot interleave half a line. The four-line banner is the one place that **re-enters**: `banner()` locks `g_console`, then calls `linef()`/`pipeline::line()` for each line, each of which locks it again. With a normal `mutex` that is a self-deadlock; `mutex_recursive` (the µOS++ equivalent of the Rust `ReentrantMutex`) lets the same thread re-lock and counts the depth. Pipe 6 ("banner printed 4 lines at console-lock depth 2") exists precisely to prove this path still works.

The distinction between `mutex` and `mutex_recursive` matters elsewhere too: the **device** is guarded by a plain `store::mutex()` (no legitimate re-entrancy — a thread must never take the SPI lock twice), while the **console** is recursive. Mixing the two up would either deadlock the banner or silently allow a second, non-owning thread to barge into the device.

5. ISRs may only post; the three SPI handlers obey that rule

The three device handlers live in the **BSP** (`test/boards/weactf411/src/spi.cpp`), not in the example, because the vector table is a board property. In µOS++ a device IRQ handler is just a strong C-linkage

function whose name matches a weak alias in the device vector table — there is no registry call. Each of the three (`SPI1_IRQHandler`, `DMA2_Stream0_IRQHandler`, `DMA2_Stream3_IRQHandler`) does exactly two things: clear the peripheral flag(s) and `g_done.post()`. Nothing else. That is the µOS++ interrupt contract: from handler mode you may call `semaphore::post()` / `event_flags::raise()` (both end in a PendSV reschedule request) and *nothing that blocks, allocates, or takes a mutex*.

The priority rule is enforced by the port, not by convention: `semaphore::post()` asserts `port::interrupts::is_priority_valid()`, which compares the handler's NVIC priority against `OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY`. The BSP therefore programs the three IRQs at priority 5: numerically ≥ the kernel's BASEPRI level, so they are **masked while the kernel holds its lock** (and may safely signal), yet more urgent than PendSV/SysTick (both 15). The app config that pins this is `os-app-config.h` (`OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY (5)`).

6. Timed waits are ticks, and the tick is 1 ms

`os-app-config.h` sets `OS_INTEGER_SYSTICK_FREQUENCY_HZ (1000)`, and the Cortex-M port programs SysTick from `SystemCoreClock / 1000`. All µOS++ wait timeouts are in **ticks of the object's clock** (default `sysclock`), so at 1000 Hz a timeout of `N` *is* `N` milliseconds:

```
sysclock.sleep_for (pipeline::BLINK_HALF);                    // 300 ms
g_write_snapshot.timed_wait (pipeline::SNAPSHOT_WAIT);        // 400 ms
```

Two design points hang off this:

- **T4's snapshot wait is bounded by design** (spec defect #12). The spec has T4 wait five times on a *binary* snapshot semaphore that T3 posts up to eight times; posts that arrive while T4 blinks are dropped. An unbounded wait would block forever on the cycle after T3 finished. `timed_wait`'s `ETIMEDOUT` is the *ordinary* outcome once the writer is done, and T4 treats it as "nothing new", not failure. T4 always runs all five cycles.
- **The producer is paced with a total, not a per-chunk, budget** (spec defect #18). `PRODUCE_SPAN = 2400` ms is divided by the chunk count, which is a function of the image length (141 chunks on silicon, 32 under QEMU). A fixed per-chunk delay right for one machine would be eleven times wrong for the other. Without the pacing the whole file lands before T4's first blink and every snapshot reports the same finished length — which is why `check.sh` insists T4 saw **at least three distinct** file lengths.

7. `sysclock.now()` is the timestamp, and format strings must not lie

µOS++ `sysclock.now()` returns ticks (`uint64_t`) since the scheduler started. Transcript timestamps print it with `%8lu` after casting to `unsigned long`.

That casting is not cosmetic. This build links newlib **nano** printf (`--specs=nano.specs`), whose formatter does **not** implement the `%zu` length modifier. The port's first transcript was a wall of corruption — `iter zu`, file names `enczu.b6`, a garbage "cycles" count and pipe 4 failing for the wrong reason — all because one `%zu` made the formatter print `zu` literally and shifted every subsequent argument. The rule adopted: **only** `%u` / `%x` / `%lu` / `%s` and explicit `static_cast<unsigned>` / `static_cast<unsigned long>` on every

`size_t` / `uint32_t` /tick argument. The Rust port's `{:>8}` and `{:#010x}` do not have this failure mode; the C++ port's format strings are the place where the two ports differ most easily.

The BSP the app leans on

Three SPI transports, chosen by asking (probe)

`spi::probe()` clocks **one byte with chip select deasserted** — nothing is selected, so nothing reaches the flash — and keeps the first transport whose completion interrupt actually arrives within a bounded wait: **DMA2** first (SPI1 RX = DMA2 Stream0/Ch3, TX = DMA2 Stream3/Ch3), **SPI1 IRQ** (RXNE/TXE) second, **polling** last. Under QEMU, whose `netduinoplus2` models no DMA2 and delivers no SPI1 interrupt, the probe falls through to `polled`; on silicon it reports `dma`. One image, two machines.

Two µOS++-specific decisions inside:

- The **completion is the semaphore** `g_done`, posted by the handler that finishes a transfer; the waiting thread does a `timed_wait`. That makes the probe and every bulk transfer interrupt-safe and deadline-bounded, and it is why the probe can run from a thread at all.
- The register-level teardown keeps **RX as the completion carrier**, not TX: TX `TCIF` would fire a frame early while the last byte is still shifting in. After the RX stream reports `TCIF`, the driver drains `SPI_SR_BSY` before raising CS.

Flash busy-wait is bounded, because an absent part reads "busy"

`BUSY` is status-register **bit 0**, and a bus with nothing driving it reads `0xFF` — which has bit 0 set. An unbounded busy-wait on a missing part is therefore "busy forever". `flash::wait_idle` polls a bounded number of times with a spin between, then reports a timeout. It is the same reasoning as the clock fallback: **nothing on this board may spin forever waiting for a bit that a missing device would never set.**

The flatfs `Writer` copies its `Partition` — that was a real bug

`flatfs::Writer` needs to remember which partition it is appending to. The Rust `Writer` stores a `Partition` **by value**. The C++ port's first draft stored a `const Partition*` — and `Storage::create` builds its partition as a temporary (`partition(which).create(...)`), so the writer's pointer dangled the moment the statement ended. On silicon the first runs wrote the encoded file correctly but `finish()` programmed its TOC entry through the dangling pointer, so `find()` never saw it: `T5 read` fault, `dec 0 B`, `!OK`, every iteration. QEMU never caught it because QEMU uses the RAM store, which has no partitions. The fix was to make the writer hold the `Partition` **by value**, exactly as the Rust type does. It is the same failure class as the `monotonic` bug: a C++ value type silently dropped a property the Rust type carried, and only silicon (or a partition-backed QEMU) could expose it.

Two machines, one binary

The clock boot in the board BSP (`test/boards/weactf411/src/os-hooks.c`) polls each PLL ready flag a bounded number of times. QEMU never asserts `HSERDY`, so the image stays on the reset HSI at 16 MHz there; on silicon the HSE 25 MHz → PLL → 100 MHz switch lands and `SystemCoreClockUpdate()` records 100 MHz. The tick interval and the USART1 baud divisor follow automatically — no second build.

The store then decides its backend by asking the part: `JEDEC 0xef …` (Winbond) means the two real SPI-flash partitions; anything else means the RAM disk. The transcript says which one ran on its first lines.

Under QEMU

The harness registers **no** QEMU case for this board: `boards/weactf411/board.cmake` sets no `UOS_BOARD_LINKER_QEMU`, so the board gets `hwd` cases only. The fall-backs above (the `polled` transport, the RAM store, the HSI clock) are still in the code; the QEMU column under *Measuring it* is the predecessor's measurement.

On the board

From `micro-os-plus-iii-smp.git/tests`, with the board on its DAPLink probe:

```
xpm run test-spi-pipeline-hwd --config cortexm-weactf411-cmake-gcc-debug
```

The transcript is on the board's **UART** (USART1, 115200 8N1); the harness's log (`build/<config>/.hw-logs/`) holds OpenOCD and the semihosting output: the CRC witness lines (the `CRC0` / `CRC1` / `OK` triplet), which is why the runner enables semihosting before `reset run`. Expect `SPI transport dma`, `JEDEC 0xef …`, the partition layout, eight clean iterations and `RESULT: PASS`. Capture the UART to a file and check it:

```
./check.sh uart.log dma
```

`check.sh` strips `\r` itself.

The verdicts

Per iteration, checked in-program and counted into the six pipes:

1. T3 wrote exactly the expected chunk count and saw the final chunk;
2. the encoded file is exactly `encoded_total(src_len)` and the decoded file is exactly `src_len`;
3. `CRC1 == CRC0` (the CRC-32 is the standard reflected `0xEDB88320` with final inversion — the value `zlib.crc32` computes, and the one the C++ `fs-archive` reference stores);

4. T4 ran all five blink cycles, its snapshot lengths never shrank, and the longest never exceeded the writer's length;
5. the Base64 codec agrees with the RFC 4648 §10 vectors — checked once, at start-up, because a round trip alone cannot catch a codec that is wrong symmetrically;
6. the banner really did print at console-lock depth 2.

`spi-pipeline: PASS` only when all six pipes and all eight iterations are clean. `check.sh` adds the gates the program cannot assert on itself: the transport line names the expected one, all eight iteration summaries are present, and T4 saw ≥ 3 distinct snapshot lengths (so the pacing and the observer really ran).

Measuring it

Measured on the predecessor's build (on silicon the numbers depend on the image length, which is a property of the build):

	QEMU (RAM store)	Board (SPI flash)
Transport	`polled`	`dma`
Store	`RAM`, source clamped to 8000 B	`SPI flash`, the whole image
Chunks	32 (last 64 B)	149 (last 168 B for a 38056-B image)
Heap per iteration	flat (reaper returns each iteration's stacks)	flat

The `spi-pipeline: heap …` first line and the per-iteration semihosting heap lines are the leak witness: they stay level across all eight iterations because the idle thread reaps each iteration's exited threads before the next one spawns.